



ObjectRT Runtime — Implementation Notes

Method dispatch, host binding system, resolver chain, EmitDir/CacheDir.

charlie santana - Finite

2026-07-31

Contents

1. Method dispatch order	1
2. Host binding system (interface contracts)	1
2.1. <code>[IRHostBinding("Name")]</code>	1
2.2. <code>HostDispatchGenerator</code> (source generator)	1
2.3. <code>InterfaceHostResolver</code>	1
2.4. <code>ClrNativeResolver</code>	2
3. Void-native null collision	2
4. EmitDir / CacheDir	2
5. Namespace gotchas	2
6. Source files	2

1. Method dispatch order

When `Runtime.CallMethod<T>(qualifiedName, args)` is invoked, dispatch follows this chain:

1. Explicit registry (`_nativeMethods`) — fastest path, `string → Delegate`
2. `INativeResolver` chain — `ClrNativeResolver`, then `InterfaceHostResolver`
3. `IExecutor` — module function map, then `NativeCallHandler` fallback

Each resolver is tried in registration order. The first non-null result wins. The explicit registry is checked before any resolver.

2. Host binding system (interface contracts)

Three cooperating pieces provide reflection-free host dispatch — suitable for NativeAOT builds where `System.Reflection` may be trimmed:

2.1. `[IRHostBinding("Name")]`

Marks a C# interface as a script-facing host contract. Scripts call methods through the standard `call` opcode using the binding name:

```
1 [IRHostBinding("MonoGame.Screen")]
2 public interface IMonoGameScreen {
3     void Clear(int color);
4     int Width();
5 }
```

C#

```
1 call MonoGame.Screen.Clear(int32)
```

oil

2.2. `HostDispatchGenerator` (source generator)

A Roslyn incremental generator that reads every interface marked with `[IRHostBinding]` and emits `HostDispatchRegistry.Register(...)` calls with direct interface casts:

```
1 HostDispatchRegistry.Register(
2     "MonoGame.Screen", "Clear",
3     (object h, object?[] a) => {
4         ((IMonoGameScreen)h).Clear((int)a[0]!);
5         return null;
6     });
```

C#

Zero reflection at runtime — the casts are hardwired. This is what keeps host bindings working when `AllowReflection = false`.

2.3. `InterfaceHostResolver`

An `INativeResolver` that dispatches through two paths in order:

1. Source-generated hardwired adapter from `HostDispatchRegistry` — direct interface cast, no reflection.
2. Reflection fallback on the registered interface — convenient during development or when the generator is not wired into the project.

`AllowReflection = false` forces generated-only dispatch. The NesDemo frontend proves this is functional — it runs with reflection disabled.

Register implementations via:

```
1 rt.RegisterHost("MonoGame.Screen", impl, typeof(IMonoGameScreen));
```

 C#

Note: `RegisterHost<T>` infers the concrete class type, not the interface. Use the explicit `(name, impl, typeof(IFace))` overload.

2.4. ClrNativeResolver

Reflection-based resolver for CLR static methods. Registered types are discovered by method name + argument count. `AllowReflection = false` disables it (NativeAOT toggle). Caches resolved delegates per qualified name.

3. Void-native null collision

`TryResolveNative` returns `out bool resolved`. A void method returning `null` is correctly classified as “**resolved and returned null**”, not as “**unresolved**”. Resolver returning null delegate = not found. This was a hard bug: void methods looked like dispatch misses.

4. EmitDir / CacheDir

```
1 Runtime.EmitDir = "./generated"; // writes {Module}.ObjectRT.g.cs
2 Runtime.CacheDir = "./cache"; // caches compiled {hash}.dll
```

 C#

Set once at startup. `EmitDir` dumps the generated C# source for every module compiled through `ReflectionJit`. `CacheDir` saves Roslyn-compiled assemblies keyed by SHA-256 hash of module content — subsequent runs skip Roslyn entirely and just `Assembly.Load` the cached DLL from disk.

5. Namespace gotchas

- Inside namespace `ObjectRT.MonoGame`, the simple name `Runtime` binds to `ObjectRT.Runtime` (the namespace), not `ObjectRT.Runtime.Runtime` (the class). Use `using ObjectRTRuntime = ObjectRT.Runtime.Runtime;`
- `RegisterHost<T>` infers the concrete class type, not the interface. Use the explicit overload: `rt.RegisterHost("Name", impl, typeof(IFace))`.
- Generated code needs `#nullable enable` plus `#pragma warning disable` suppressions for CS8600, CS8601, CS8602, CS8603, CS8604, CS8605, CS8625 or the consuming project warns on the generated hard casts.

6. Source files

1	src/ObjectRT.Runtime/	
2	— Runtime.cs	– host, module loading, method dispatch, JIT config
3	— ClrNativeResolver.cs	– reflection-based static method resolution
4	— InterfaceHostResolver.cs	– host interface dispatch (generated + reflection)
5	— INativeResolver.cs	– resolver interface
6	— IRHostBindingAttribute.cs	– [IRHostBinding] attribute

7	— HostDispatchRegistry.cs	– generated adapter registry
8	— IRClassBindingAttribute.cs	– [IRClassBinding] for script proxies
9	— IRMethodBindingAttribute.cs	– [IRMethodBinding] for method overrides
10	— IRRuntimeBinding.cs	– late-bound call handle
11	— ProxyRegistry.cs	– source-generated proxy registration
12	— JitMode.cs	– interpreter/reflection/LLVM enum